

Network Defense (Fall 2024) – Weekly Portfolio

Course Overview: *Network Defense (CSC-7303)* was a hands-on graduate-level course focused on protecting enterprise networks. Over 12 weeks, I built and secured a multi-VM lab environment, applied defensive tools and strategies, and completed projects simulating real-world cyber defense scenarios. Each week introduced new concepts and practical exercises, from setting up firewalls and domains to implementing intrusion detection, threat hunting, and incident response. Below is a week-by-week breakdown of what I learned and accomplished.

Week 1: Lab Environment Setup & Network Fundamentals

Summary: I set up the foundational lab network using VirtualBox and a *pfSense* virtual router. This included configuring network interfaces and VM instances for various operating systems (Windows and Linux) and verifying connectivity. We discussed basic network security principles and ensured the lab's virtual network mimicked a typical corporate setup (internal LAN, and a gateway firewall). Key networking concepts like NAT vs. host-only networking were applied to simulate internet and intranet segments.

- **Virtual Network:** Created an isolated virtual network with a *pfSense* firewall router VM as the gateway. Configured NAT (for internet access) and Host-Only networks (for internal communication) to segregate external vs. internal traffic 🟡.
- **Systems Deployed:** Installed Windows Server 2022, Windows 11, Ubuntu 24, CentOS 9, and Kali Linux VMs with appropriate static IPs (e.g. Win11 at 192.168.1.100) 🟡. Enabled DHCP on the server/router and verified all machines could ping each other and the gateway.
- **Firewall Basics:** Initial *pfSense* configuration with default *deny-all inbound* rules and *allow internal outbound* policies. Learned how a firewall “**monitors and filters network traffic based on predefined security policies**” 🟡, implementing basic packet-filtering rules.
- **Networking Fundamentals:** Reviewed TCP/IP, subnets, and common ports. Ensured each VM's network adapter was correctly set (e.g. internal VMs on the Host-Only LAN, Kali on both LAN and NAT to simulate an attacker with internal and external access). Resolved early connectivity issues by correcting adapter settings and IP configurations 🟡.

Week 2: Securing Windows & Linux Endpoints (Baseline Hardening)

Summary: With the network up, I focused on baseline hardening of the endpoint VMs. This included updating OS packages, configuring host firewalls, and basic security policies on Windows and Linux. I ensured Windows 11 and Linux could integrate later with a domain. We also explored host-based security features and prepared the environment for Active Directory deployment.

- **Windows Hardening:** Enabled Windows Defender firewall on Windows 11, set inbound rules to restrict unnecessary services, and applied local security policies (password complexity, account lockout). Explored features like BitLocker and UAC as additional host protections.

- **Linux Hardening:** On Ubuntu and CentOS, configured `ufw`/`firewalld` for basic firewall rules and ensured secure SSH settings (changed default SSH port, disabled root login). Updated all packages and created a regular patch management routine.
- **Accounts & Permissions:** Created standard user accounts on each OS to avoid running as Administrator/root. Practiced principle of least privilege by adjusting file permissions and using sudoers configuration on Linux.
- **Lab Verification:** Tested connectivity and firewall rules – for example, confirming that Windows firewall blocked ping requests from Kali until allowed. These steps established a secure baseline for each machine's local defenses before domain integration.

Week 3: Active Directory and Group Policy Implementation

Summary: This week I set up **Active Directory (AD)** on Windows Server 2022 and enrolled both Windows 11 and Linux clients into the domain. This involved promoting the server to a Domain Controller, creating an AD forest (we humorously named it “evil.corp” domain), and configuring DNS. We then applied Group Policy Objects (GPOs) and Windows security features like AppLocker to centrally manage the environment's security. The lab consisted of a Server 2022 domain controller with clients running Windows 11, Ubuntu 24 (using LDAP/SSSD for domain join), and CentOS.

- **Domain Controller Setup:** Installed AD Domain Services on Windows Server and created a new domain “evil.corp” (as an example forest). Set up DNS and verified the domain controller's connectivity. The AD structure (forest, domain tree, OUs) was reviewed 🟡🟡.
- **Domain Joins:** Successfully joined the Windows 11 client to the AD domain (allowing central login). For Linux, configured **SSSD (System Security Services Daemon)** to integrate Ubuntu with AD for authentication 🟡🟡. This demonstrated cross-platform domain integration, allowing Linux users to log in with domain credentials.
- **Group Policy Management:** Used the Group Policy Management Console (GPMC) to create and link GPOs. For example, enforced password complexity and interactive logon messages via GPO. Verified that the policies propagated to the Windows 11 client (e.g., minimum password length and lockout rules applied centrally).
- **Application Control:** Implemented **AppLocker/Windows Defender Application Control (WDAC)** via Group Policy on the Windows 11 machine. This restricted unauthorized applications – for instance, we created a rule to block a disallowed game by publisher name. Learned that “application control from the Server 2022 will only work on Windows 11; Ubuntu and CentOS required manual controls” 🟡🟡. This highlighted the need for different tactics on Linux (using file permissions or AppArmor/SELinux for similar restrictions).

Week 4: Network & Perimeter Security – Firewalls and IDS

Summary: Focus shifted to **perimeter defense**. We configured network firewall rules on pfSense and introduced Intrusion Detection/Prevention Systems. I enabled a Snort/Suricata package on pfSense to serve as a **Network Intrusion Detection System (NIDS)**. Meanwhile, we deployed **Wazuh** (an open-source fork of OSSEC) on endpoints as a **Host Intrusion Detection System (HIDS)**. This combined approach gave experience in both network-level and host-level threat detection.

- **Firewall Rule Tuning:** Added granular pfSense rules for inbound/outbound traffic. For example, allowed web traffic to the DMZ server while blocking unused ports, and set up outbound rules to

block certain external IP ranges. Reinforced how a firewall “inspects each packet against security rules, permitting or blocking based on policy” ().

- **Intrusion Detection (NIDS):** Configured **Suricata** on pfSense to monitor network traffic. Loaded Emerging Threats rulesets to detect malicious patterns. We generated test traffic (e.g. a crafted *DNS query indicating a Kali hostname*) and saw the NIDS trigger an alert. One alert flagged a DHCP request from our Kali box, categorized as *“Possible Kali Linux hostname in DHCP request – Potential Corporate Privacy Violation”* (), which would warn a defender of an unauthorized penetration testing device on the network.
- **Host-Based Monitoring (HIDS):** Installed **Wazuh agents** on Windows and Linux hosts. Wazuh aggregated system logs and monitored for suspicious events (file integrity changes, malware signatures, etc.). We tested Wazuh by simulating a malware drop and verifying the alert on the Wazuh dashboard. The instructor noted that we should all have Wazuh set up and reporting in our lab environment by this point ().
- **Lab Activity:** This lab was challenging – effectively configuring a full IDS/IPS. Troubleshooting was required (e.g., enabling the correct rule to detect a test attack). It emphasized that in practice *“you’re allowed to struggle”* through complex setups and learn from failures (). By the end, both NIDS and HIDS were operational, giving us a layered defense: network sensors to catch attacks in transit and host agents to catch those that reach a system.

Week 5: Multi-Factor Authentication, Segmentation & Access Control

Summary: We covered advanced defensive strategies: **Multi-Factor Authentication (MFA)**, network segmentation, role-based access control, and secure remote access. These lectures were more conceptual but reinforced how to strengthen security beyond basic firewalls. In the lab, we continued refining our AD environment – e.g., enabling MFA for domain accounts (using Windows Hello and authenticator apps) – and experimented with virtual network segmentation on our VirtualBox network.

- **Multi-Factor Authentication:** Discussed adding MFA to critical systems. On Windows, we explored **Windows Hello** (biometrics/PIN) and integration with Microsoft Authenticator. Requiring a second factor significantly reduces account compromise risk – *“stolen passwords alone won’t allow unauthorized access”* if MFA is enabled (). I enabled MFA for the domain admin account as a proof of concept.
- **Network Segmentation:** Learned about creating VLANs or separate subnets to isolate sensitive network zones. In our lab, VirtualBox’s limitations meant we could not fully implement VLAN tagging, but we conceptually created a *“DMZ network”* for the public-facing server. The instructor noted we should still **“know how to set them up”** and that more advanced classes (like those with Palo Alto firewalls) would cover VLANs in depth ().
- **Role-Based Access Control (RBAC):** Reviewed how to assign permissions based on roles (e.g., admin, user, guest). In AD, I organized users into groups and applied group-based permissions on shared resources. This ensures principle of least privilege – each role gets only the access required.
- **Secure Remote Access:** Covered VPNs and secure protocols for remote admin. We configured *OpenVPN* on pfSense for remote connectivity and tested RDP into the Windows server over the VPN. Also discussed hardened SSH: using key-based authentication and MFA for any SSH access to servers.
- **Key Takeaway:** Defense-in-depth includes not just tools but policies. By combining **MFA, network segmentation, RBAC, and secure remote access**, we add redundant layers of security. This week

tied together those strategies – “covering multi-factor auth, firewall rules, a bit of network segmentation, RBAC, remote access, security scanning” all at once  .

Week 6: Vulnerability Scanning and Penetration Testing Basics

Summary: We shifted perspective to learn how attackers operate, so we as defenders can identify weaknesses. Using our Kali Linux VM (attack box), I performed network reconnaissance and vulnerability scans on the lab environment. We employed tools like **Nmap** for enumeration, **Hydra** for brute-force password testing, and **OpenVAS** (Greenbone Vulnerability Manager) for automated vulnerability scanning. This ethical hacking practice revealed misconfigurations and vulnerabilities in our setup that we then learned to mitigate.

- **Network Enumeration (Nmap):** Scanned the lab network (e.g., `192.168.1.0/24`) to discover hosts and open ports. Mapped the attack surface – for instance, found that our Windows Server had TCP ports 53 (DNS), 3389 (RDP) open, etc. This taught how adversaries identify targets and how we should minimize exposed services.
- **Password Brute Forcing (Hydra):** Used Hydra to attempt a brute-force login on an intentionally weak account. For example, we tested SSH on the Ubuntu box with a small password list. This demonstrated the importance of strong passwords and account lockout policies (our earlier GPOs for account lockout thwarted the brute force after 5 attempts, which we verified in Windows security logs).
- **OpenVAS Vulnerability Scan:** Deployed **OpenVAS** on Kali to run an authenticated vulnerability scan of the Windows 11 and Ubuntu systems. The scan identified several vulnerabilities and misconfigurations. Notably, it flagged that Windows 11 was missing certain patches and that default SMB settings could be hardened. On Linux, it found an outdated Apache version on our web VM. This reinforced proactive scanning to find and fix issues before attackers do.
- **Results & Mitigation:** We analyzed OpenVAS reports, which listed findings by severity. We then took steps to mitigate: applying Windows updates, disabling old SMB protocols, and updating Linux packages. The “*Vulnerability Testing*” report I compiled showed the process and results. By understanding these tools, I learned how attackers find weaknesses and how to prioritize remediating them.

Week 7: Windows Threat Hunting and Attack Simulation

Summary: This week delved into detecting and responding to host-based attacks, particularly on Windows. We simulated a breach by performing a controlled **privilege escalation** on the Windows 11 VM and analyzing the artifacts left behind. Using tools like *Mimikatz* (to simulate credential theft) and generating suspicious activities, we practiced identifying such events via logs and our HIDS (Wazuh) alerts. The focus was on **Windows event analysis** and ensuring our detection tools were correctly tuned.

- **Privilege Escalation Lab:** Conducted a scripted attack on the Windows VM (with prior permission). For example, we used a known vulnerability to gain admin rights from a standard user account. This was part of “*Part 6: Escalate Windows VM*” in the project. The exercise let us see how an attacker might elevate privileges after an initial compromise.
- **Log Forensics:** After the simulated attack, we combed Windows Event Viewer and Wazuh logs for signs of the breach. We found events like Windows Security Log ID 4673 (sensitive privileges used)

and Wazuh triggered alerts on suspicious process execution. This validated our monitoring setup – Wazuh caught the privilege escalation attempt in real time.

- **HIDS Tuning:** Fine-tuned Wazuh rules to reduce noise and focus on critical incidents. We enabled specific rules to alert on common post-exploit behaviors (e.g. new user added to Administrators group, registry run key modifications).
- **Reporting:** Documented the incident in a mini report describing how we detected it and what steps would contain and eradicate such an intrusion. Practicing this incident response reporting was valuable for real-world readiness.
- **Security Planning:** In tandem, I worked on a formal **Security Plan** document summarizing the network's defenses and remaining gaps. This plan (submitted as part of the midterm project) outlined security policies and improvements moving forward. It was essentially an assessment of our lab's security posture after weeks of hardening and where to focus next (for example, suggesting to implement centralized SIEM logging for better visibility).

Week 8: Midterm Project Synthesis and Presentation

Summary: Around this time, the **Midterm Project** was due. It comprised all the configured systems and defenses to date, plus documentation. I prepared a comprehensive formal report that compiled the work from Weeks 1–7, including diagrams, configurations, and findings. We also had a review session, and each student summarized their network defense setup. This was an opportunity to reflect on how each component interlocks to provide defense in depth.

- **Firewall & AD Diagram:** Created network diagrams showing the pfSense firewall, the domain controller, and network segments (LAN, DMZ). This illustrated the architecture built in the first half of the course.
- **Midterm Documentation:** The submission included multiple parts: (1) Firewall setup, (2) Windows/Linux baseline configs, (3) AD setup, (4) Wazuh deployment, (5) Vulnerability assessment results, (6) Windows escalation test, (7) Security improvement plan, and (8) Formal summary report. Each part was a detailed write-up with screenshots and analysis. For example, the **Formal Report** executive summary highlighted key learnings like the importance of patch management and log monitoring.
- **Findings:** One striking finding was from the Wazuh compliance scan of Windows 11 against CIS benchmarks. It reported only **32% compliance** (263 out of 395 checks failed) in default state . Critical policies such as *minimum password length* and *password complexity* were flagged as failed, which we had begun to address via Group Policy . Including these findings in the report demonstrated the tangible improvements made (e.g., after GPO enforcement, those password policies would be compliant).
- **Presentation:** I presented a short overview of my network defense lab, walking through how an attacker would be thwarted at each layer. For instance: perimeter firewall blocks unused ports, NIDS alerts on suspicious traffic, HIDS catches anomalous host behavior, and regular vulnerability scans ensure systems stay hardened. This holistic understanding was a big outcome of the midterm project.

Week 9: Container Security and DMZ Vulnerability Scanning

Summary: Building on network segmentation, we introduced a **Dockerized DMZ environment**. I set up a *DMZ Linux server* hosting intentionally vulnerable web applications (DWVA – Damn Vulnerable Web App) in Docker containers. Then, I deployed OpenVAS as a container to scan those services. This lab taught how to

secure containerized environments and scan them for weaknesses, reflecting modern enterprise setups where applications often run in containers.

- **Docker in the Lab:** Installed Docker on the designated DMZ server (an Ubuntu VM). Pulled the latest `vulnerables/web-dvwa` image and ran the DVWA web app container (mapped to host port 8080). This provided a sandbox web app with known vulnerabilities for testing.
- **OpenVAS Container:** Instead of Kali's installation, we used an **OpenVAS Docker container** via `docker-compose`. Set this up on a separate VM (or Kali itself) to act as a dedicated scanner. Following steps, we ran `docker-compose up -d` to launch the OpenVAS service in a container. This approach mimics how enterprises might use containerized security tools.
- **Scanning the DMZ:** Configured the OpenVAS container's web UI (Greenbone Security Assistant) and created a scan target for the DMZ server's IP. Launched a vulnerability scan against the DVWA app and the underlying system. The scan identified the intentionally vulnerable configurations in DVWA (e.g., SQL injection, weak admin password) and also any host-level issues on the DMZ server.
- **Container Security Lessons:** Discussed best practices for container security – e.g., limiting container privileges, regularly updating images, and using container-specific scanning tools. The lab showed how a compromised DMZ web app could be discovered via scanning and reinforced isolating such apps in containers to protect the internal network.
- **Outcome:** Gained experience in using Docker for both deploying targets and security tools. This reflects a realistic scenario of deploying a service in a DMZ and using an internal scanner to continuously assess it for vulnerabilities. It emphasized that even containerized apps need patching and monitoring.

Week 10: Advanced Tools – Automated Hardening, OpenVAS & Metasploit

Summary: We returned to Kali Linux to configure advanced offensive tools and also ensure the Kali system itself was secured and automated. I went through a “Hardening Kali” exercise where I scripted automatic updates and task scheduling on Kali. We then installed **OpenVAS on Kali** (this time the full version on the OS) and set up **Metasploit** to exploit a test target (Metasploitable2 VM). This week bridged defensive hardening with offensive testing, showing the full cycle of finding and exploiting vulnerabilities and then fixing them.

- **Automating Updates (Hardening Kali):** Wrote a Bash script `kali_update.sh` to automatically update and upgrade Kali packages, and set it to run daily via cron. This ensures even our penetration testing machine stays up-to-date and secure from its own vulnerabilities. Configured the cron job to run at 2 AM each day, demonstrating scheduling of maintenance tasks.
- **Securing Services on Kali:** Enabled and started the `cron` service on Kali to ensure scheduled tasks execute. Also hardened SSH on Kali (even though it's an attacker box, we treat it as any system that shouldn't be left with defaults). These steps underline that any system, even tools, should be maintained securely.
- **Installing OpenVAS on Kali:** Instead of using the container again, we installed OpenVAS natively to learn the setup process. This involved adding repositories, installing PostgreSQL, and running `gvm-setup` and `gvm-start` to initialize Greenbone Vulnerability Manager. We encountered and solved connectivity issues with the Greenbone feed (for vulnerability updates) by adding the

correct GPG keys and repositories [1]. Successfully brought up the Greenbone Security Assistant web UI on Kali and verified it was operational (accessible at <https://127.0.0.1:9392>) [2].

- **Metasploit Exploitation:** Set up Metasploit Framework on Kali and targeted a known vulnerable VM (*Metasploitable2*, a purposely vulnerable Linux). We exploited a vulnerability (e.g., VSFTPD backdoor) to gain a shell on Metasploitable2. This demonstrated the process attackers use to exploit and the kind of evidence it generates. We then used Metasploit's post-exploitation modules to gather information (password hashes, etc.). In the documentation, I noted how Metasploit was configured and the outcome of the exploitation [3].
- **Analysis of Results:** After exploiting Metasploitable2, we treated it as an incident: what logs were generated, how to detect it, and how to patch the exploited flaw. This double role – attacker and defender – gave insight into how a misconfigured service can lead to a full compromise. It reinforced the need for regular vulnerability scanning and prompt patching, which ties back to our use of OpenVAS.

Week 11: Network Monitoring, SIEM, and Log Analysis

Summary: As the course neared conclusion, we focused on **continuous monitoring and log analysis** – essentially how to operationalize all our defenses. We discussed Security Information and Event Management (SIEM) tools and practiced analyzing logs from various sources (firewall, OS event logs, IDS alerts) to spot suspicious activity. This week tied together the detection engineering aspect: making sense of the data our security tools produce.

- **SIEM Concepts:** Learned about SIEM solutions for centralized log management. The instructor introduced tools like **Splunk**, **ELK (Elasticsearch/Logstash/Kibana)**, and **Graylog**, which aggregate logs from across the network for analysis [4]. We didn't deploy a full SIEM due to time, but we used Wazuh (which has SIEM-like capabilities) to illustrate how log data flows from agents to a dashboard.
- **Log Types:** Reviewed the key log categories: firewall logs (connections blocked/allowed), authentication logs (login successes/failures), system logs (services starting/stopping, errors), and network flow records [5] [6]. I looked at pfSense firewall logs in real-time and Windows Event Viewer security logs side by side to understand normal vs. abnormal patterns.
- **Log Analysis Exercise:** We were given sample logs (and also used some from our own lab incidents) to analyze. One exercise was to identify a port scan in firewall logs – we observed a single host attempting connections to many ports in a short time, a clear scan fingerprint. Another was analyzing Windows logs for multiple failed logins followed by a success, indicating a possible brute force login that succeeded.
- **Tool Usage:** Although we had used Wazuh and Suricata's interfaces, here we practiced raw log analysis. For instance, using the Windows Event Viewer filters to find all events related to account lockouts or using Linux `grep` on syslogs for keywords like "Failed password". This sharpened our ability to parse logs without always relying on GUI tools.
- **Key Takeaway:** *"Securing network operations requires mastering log monitoring and traffic analysis"* [7]. This motto from the lecture emphasizes that all the security controls we set up produce data – and it's on us to monitor that data continuously. I configured email alerts for certain Wazuh triggers (like high-severity alarms) as a practical step toward an incident alerting system. By the end of the week, I had a clearer picture of how to integrate these tools and logs into an operational security monitoring program.

Week 12: Capstone – Final Project & Reflection

Summary: In the final week, we started the **Final Project**, which was effectively an individual capstone. The task was to apply “everything learned” by securing a set of provided virtual machines and writing a professional report on the process. The instructor gave us images of systems with unknown vulnerabilities/misconfigurations, and our job was to harden them, implement defenses, and then demonstrate the improvements with evidence. Although most of the work would be done in the following two weeks, this week’s session was a kickoff and also a reflection on the course as a whole.

- **Final Project Kickoff:** We received instructions for the final: a scenario with multiple VMs (e.g., a Windows server and a Linux server) that we had to secure as if we were corporate network defenders. We were allowed to work in pairs. All the skills from prior weeks were needed – setting up firewalls, joining to domain, patching, enabling logging, etc. Essentially, *“set up the VMs and secure them applying everything you’ve learned in this course”* ☐.
- **Planning & Requirements:** We reviewed the checklist of requirements (install certain defenses, enforce policies, demonstrate an attack before and after hardening, etc.). The instructor provided passwords for the VMs and mentioned that initially they were intentionally vulnerable. Over the next two weeks, we would configure these machines securely and then submit a report with screenshots of applied controls and logs of any blocked attacks ☐ ☐.
- **Course Reflection:** We discussed as a class what worked well and what we found challenging. Many noted the steep learning curve of tools like Wazuh or the complexity of configuring Active Directory and Linux integration. The instructor acknowledged this and encouraged us that struggling through labs is part of the learning process ☐. By now, we had effectively built a mini enterprise network with layered defenses from scratch – a valuable simulation of a real-world network defense project.
- **Tools Recap:** We revisited each major tool and technique: pfSense (firewall/NAT), Windows AD (central auth), Suricata IDS, Wazuh HIDS, OpenVAS scanner, Metasploit, Wireshark, etc. Notably, I had become comfortable with network traffic analysis using **Wireshark and tcpdump** – earlier in the course we used these to inspect packet details and verify what our IDS was alerting on. *“I’ve already showed you guys Wireshark and TCPdump... to further illustrate getting logs from multiple places”* ☐ was a reminder of how those packet-level tools complemented our higher-level log analysis.
- **Next Steps:** The formal portfolio and final report from this project would serve to demonstrate to prospective employers the practical skills gained. I drafted outlines for my final portfolio (much of which is reflected in the above weekly breakdown). By completing the final project, I would have a comprehensive artifact showing I can design, implement, and manage a multilayered network defense, as well as communicate my process in writing.

Conclusion

Over the span of this course, I progressed from basic network setup to advanced security operations. Each week added a layer to the overall defense-in-depth strategy. I configured firewalls, learned to administer Active Directory, deployed intrusion detection both on network and hosts, performed vulnerability assessments, and practiced incident response. I also completed professional-style documentation (business case, security plan, formal reports) demonstrating my ability to analyze and communicate security posture and improvements.

This intensive hands-on experience gave me practical proficiency with tools like **pfSense (firewall)**, **Wireshark (traffic analysis)**, **Suricata/Snort (NIDS)**, **Wazuh/OSSEC (HIDS)**, **OpenVAS (vulnerability**